Team 13 Cars: Autonomous Driving Project Report

Introduction to ROS SS25

Prof. Markus Ryll

TUM School of Engineering and Design

Technical University of Munich

Team Member:
Shijie Zhou,
Richard Viegener,
Wenxuan Fu,
Wenkai Xiao,
Wenjin Zhang.

Table of Contents

Table of Figures

1. Introduction

This project was carried out as part of the "Introduction to ROS" course at the Chair of Autonomous Aerial Systems, Technical University of Munich, during the Summer Semester 2025 under Prof. Markus Ryll. For the final assignment, students were tasked with developing an autonomous driving system within a Unity-based urban simulation, enabling a vehicle to navigate the city streets without veering off the road and to correctly recognize and obey traffic signals, thereby demonstrating a fully functional urban autonomous driving scenario.

2. Overview of the project

The Unity simulator continuously outputs sensor data from cameras, LiDAR, and other sources. The perception module first processes this data to generate point clouds, build a 3D voxel grid (projected into a 2D occupancy map), and detect traffic light states, then passes these results to the decision module. The decision module combines the occupancy map, traffic light status, and the vehicle's global pose (via tf2) to issue "stop on red, go on green" commands, which it sends to the trajectory planning module. The trajectory planner uses the predefined global way-points to compute a sequence of desired poses and speed targets. Finally, the control module compares the desired versus actual poses and velocities to calculate steering, throttle, and brake commands, closing the perception-decision-planning-control loop.

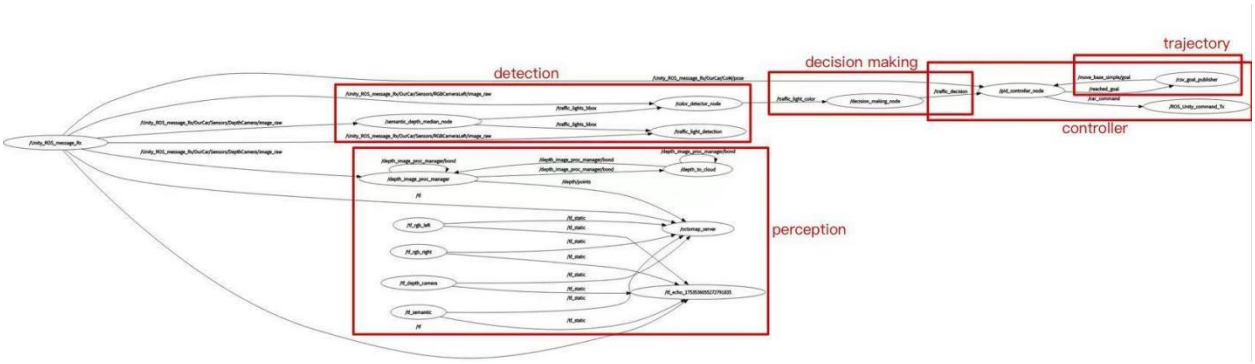

Figure 1: Overview of the process

3. Perception:

This perception module establishes a complete TF tree to align the vehicle base (OurCar/INS) with multiple onboard sensors, including the depth camera, left/right RGB cameras, and the semantic camera. While the transform from world to OurCar/INS is provided by the Unity simulation, this module is responsible for broadcasting static transforms from OurCar/INS to each sensor using tf2_ros/static_transform_publisher—ensuring valid timestamps and resolving TF synchronization issues.

The system then uses depth_image_proc to convert raw depth images and camera intrinsics into real-time 3D point clouds (/depth/points). These are passed to octomap_server to build a 3D voxel map, which is incrementally projected into a 2D occupancy grid (/projected_map) using height filters to focus on a robot-centric slice of space.

To support real-time operation, the module carefully avoids nodelet manager conflicts between perception.launch and octomap.launch, and properly configures key OctoMap parameters like resolution and projection bounds. It also provides recommended RViz visualizations for TF frames, point clouds, 3D voxel maps, and 2D navigation grids.

In summary, this module delivers a stable, real-time perception pipeline that transforms raw simulated sensor data into planning-ready maps—directly supporting downstream modules such as navigation, SLAM, and obstacle avoidance.

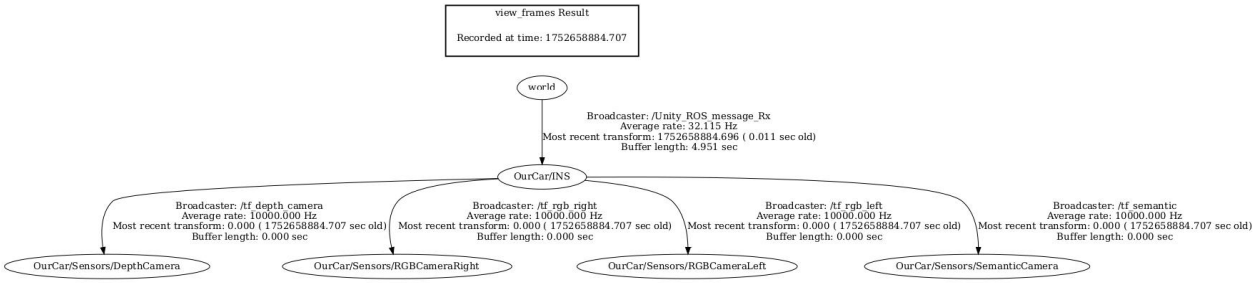

Figure 2: TF tree

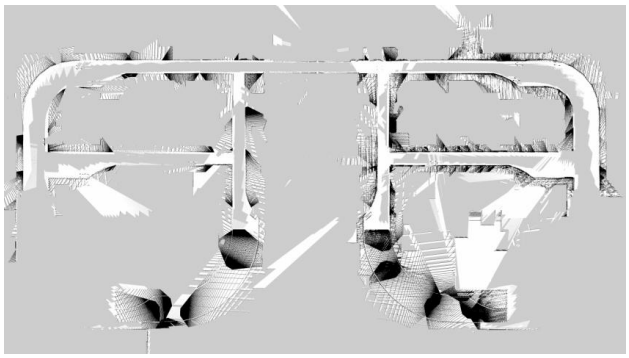

Figure 3: 2D Occupancy grid /Projected map

4. Traffic light detection

4.1 Semantic Camera Localization and Scale Normalization

First, the system detects the pixel locations and bounding boxes of all traffic lights in the image using a semantic camera. However, because the semantic camera's depth estimation scale does not match that of the depth camera and the RGB camera, scale normalization of the semantic camera's detections is required for accurate alignment.

4.2 Projecting Pixel Coordinates into the RGB View

The semantic camera and the depth camera are co-located and thus share the same extrinsic parameters; the RGB camera, however, has a fixed translational and rotational offset. The system first measures the distance of each traffic-light target point with the depth camera, then uses the pinhole camera model along with intrinsic parameters (focal length, principal point, etc.) from the /camera_info topic to back-project the depth camera's pixel coordinates into 3D space. Next, using the RGB camera's extrinsics relative to the depth camera, these 3D points are re- projected onto the RGB image plane to obtain the precise RGB pixel regions.

4.3 HSV Color-Space Filtering for State Recognition

Once each traffic light's pixel region in the RGB image is obtained, the final node performs color analysis on that region. It converts the image from RGB to HSV color space, then applies preset HSV ranges to filter red, yellow, and green light distributions, calculating the proportion or brightness of each.

Ultimately it determines the current state of the traffic light (red/yellow/green) and publishes the corresponding message for downstream decision-making modules.

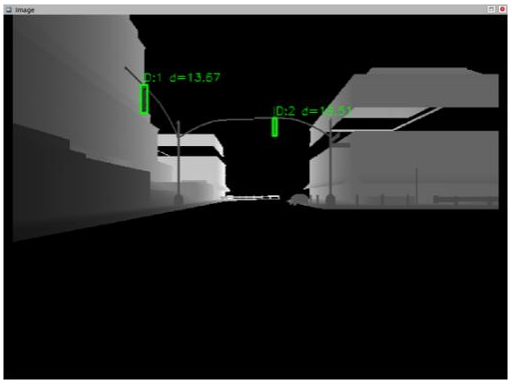

Figure 4: Depth Camera *Figure 5: Distance to Traffic Light*

5. Trajectory Planning

5.1 Path Recording Mechanism

During autonomous driving simulation, the path recording module performs sampling at a fixed time interval (e.g., every 0.5 seconds) to log the current state of the robot or vehicle. Each sampled point includes an auto-incremented index, timestamp, position (x, y, z), yaw angle, and velocity. These points are stored sequentially in memory as a complete trajectory. When the node shuts down, all recorded data is automatically exported to a CSV file for further analysis or playback.

	index	timestamp	x	y	z	yaw	velocity
2	0	1.75279e+09	-62.8002	-12.2438	0.837984	1.5707	0.285846
3	1	1.75279e+09	-62.8004	-12.0695	0.8313	1.57067	0.399432
4	2	1.75279e+09	-62.7991	-11.8434	0.82496	1.56909	0.406494
5	3	1.75279e+09	-62.7936	-11.6486	0.822889	1.56544	0.376885
6	4	1.75279e+09	-62.7907	-11.4565	0.822477	1.56402	0.417986
7	5	1.75279e+09	-62.7874	-11.2142	0.823943	1.56277	0.537955
8	6	1.75279e+09	-62.7825	-10.8979	0.827497	1.56136	0.8642
9	7	1.75279e+09	-62.7742	-10.4131	0.827536	1.55872	1.01929
10	8	1.75279e+09	-62.7636	-9.87178	0.825058	1.55629	1.05756
11	9	1.75279e+09	-62.7523	-9.35015	0.823789	1.55454	1.03907
12	10	1.75279e+09	-62.7393	-8.81089	0.823457	1.55173	1.03288
13	11	1.75279e+09	-62.7269	-8.33982	0.823303	1.54932	1.01981
14	12	1.75279e+09	-62.7121	-7.83129	0.823423	1.54685	1.01756

Figure 6: Path-Point record

5.2 CSV Export and Post-Processing

To enable target tracking control, the recording node does not publish the entire path at once, but instead sends each sampled point individually as a geometry_msgs/PoseStamped message on the /reached_goal topic. Each message contains the timestamp, coordinate frame (e.g., map), and the current 3D position. The control module subscribes to this topic and receives one goal at a time, using it as the immediate tracking target. Based on the current goal, it computes motion commands using a control algorithm (e.g., PID). The controller does not need to know the full trajectory—it simply follows the goal currently provided via /reached_goal.

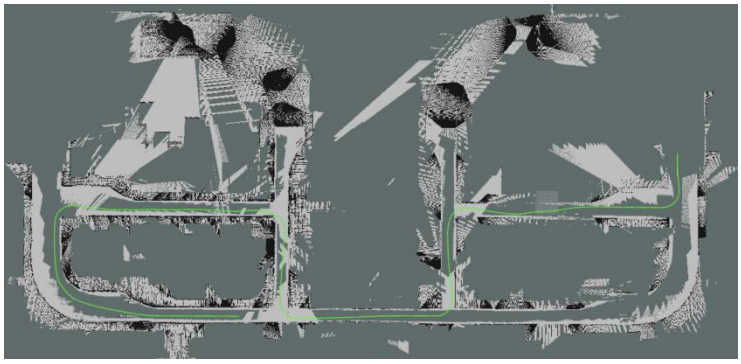

Figure 7: Trajectory

6. Decision Making

The dummy_controller package implements a ROS node that subscribes at 20 Hz to the traffic_light_color topic, receives messages of the form “id:state:confidence”, and extracts the traffic-light color and distance. When the detected distance is less than 20 m, it maps the state field to a driving command-publishing “BRAKE” for red and “DRIVE” for green; if the distance exceeds the threshold or the state is unknown, it refrains from publishing a decision or publishes ”UNKNOWN”. Finally, the node publishes the decision result to the /traffic_decision topic for downstream control modules to apply the appropriate speed control strategy (stop, accelerate, drive, or do nothing).

Figure 8: Decision Making

7. Controller

7.1 Node Functionality

The “Dummy_controller” node, in simulation, subscribes to vehicle position, yaw angle, target waypoints from the motion_planning module, and traffic light status; it uses a PID controller to compute and publish throttle, steering, and brake commands in real time, forcibly braking on red lights or upon receiving a stop signal from the decision-making node; after reaching each waypoint it reports the index to request the next target, and upon reaching the final waypoint it applies a permanent brake—providing closed-loop autonomous driving control for path tracking, traffic-light compliance, and multi-waypoint navigation.

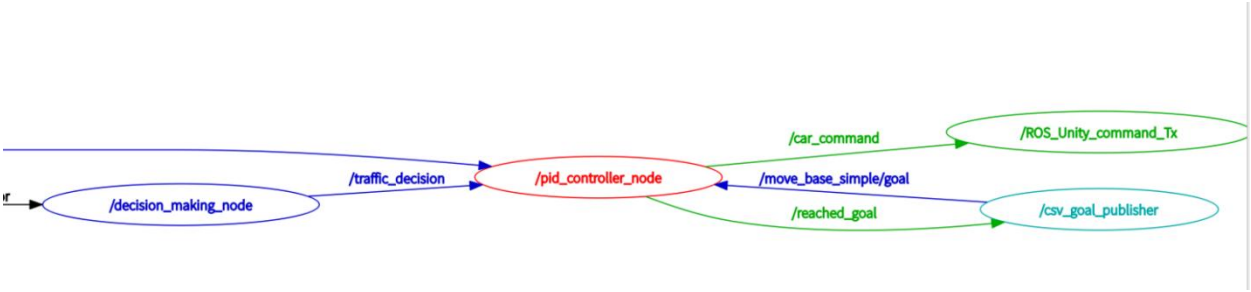

Figure 9: Subscriptions and publications of the dummy_controller node

7.2 PID Controller

The control logic is implemented by two cooperating PID controllers: a speed PID computes throttle or brake commands based on the difference between the target waypoint's world coordinates (X, Y) and the vehicle's current position to precisely regulate speed; a steering PID outputs the steering angle based on the error between the vehicle's current heading and the target point's relative yaw angle. Both controllers integrate past (integral), present (proportional), and predicted (derivative) error information, and by tuning the P, I, and D gains, the system achieves fast response, stability, and high precision—providing an optimal balance between path tracking and steering control.

8. Responsibility

Perception: Shijie Zhou (with help from Richard Viegener, Wenjin Zhang)
Traffic Light Detection: Richard Viegener
Controller: Wenxuan Fu
Decision Making: Wenkai Xiao
Trajectory Planning: Wenjin Zhang (with help from Richard Viegener)
Document and PowerPoint: Wenkai Xiao